GROUP A — Very Short Answer Questions (2 × 10 = 20 Marks)

Q1. Define data wrangling. [2 marks]

Data wrangling (also called data munging) is the process of cleaning, transforming, and mapping raw data into a more usable format for analysis. It involves handling missing values, removing duplicates, correcting errors, and reshaping data structures so analysts can derive meaningful insights.

Q2. What is an aggregation function?

An aggregation function computes a single summary value from a collection of data. Examples include sum(), mean(), count(), min(), max(), and std(). In Pandas, these are used with groupby() to summarize grouped data.

Example: df.groupby('category')['sales'].sum()

Q3. How to Manipulate and Create Categorical Variables? [2 marks]

In Pandas, categorical variables can be created and manipulated using pd.Categorical() or by converting a column with .astype('category'). Categories can be renamed, ordered, or added using .cat accessor methods.

```
df['grade'] = pd.Categorical(['A','B','C','A'], categories=['A','B','C'], ordered=True)
df['grade'].cat.codes  # convert categories to integer codes
```

Q4. Explain Hierarchical Indexing. [2 marks]

Hierarchical Indexing (MultiIndex) in Pandas allows multiple index levels on a single axis, enabling representation of higher-dimensional data in 1D (Series) or 2D (DataFrame). It supports partial indexing and cross-section selection.

```
idx = pd.MultiIndex.from_tuples([('Nepal','A'),('Nepal','B'),('India','A')])
s = pd.Series([10, 20, 30], index=idx)
```

Q5. What built-in data types are used in Python? [2 marks]

Python's built-in data types include:

-Numeric: int, float, complex

-Sequence: str, list, tuple, range

-Mapping: dict

-Set: set, frozenset

-Boolean: bool

-Binary: bytes, bytearray, memoryview

-None: NoneType

Q6. How are data analysis libraries used in Python? Common libraries? [2 marks]

Python data analysis libraries are imported at the top of a script and provide optimized, high-level functions for data manipulation, statistics, and visualization. Key libraries:

-NumPy — fast numerical computing with N-dimensional arrays

-Pandas — DataFrame-based data manipulation and analysis

-Matplotlib — 2D plotting and visualization

-Seaborn — statistical data visualization built on Matplotlib

-SciPy — scientific and statistical functions

-Scikit-learn — machine learning algorithms

Q7. Difference between lists and tuples in Python? [2 marks]

List	Tuple
Mutable (can be changed)	Immutable (cannot be changed)
Defined with [ ]	Defined with ( )
Slower performance	Faster performance
Used for homogeneous data	Used for heterogeneous/fixed data
Has more built-in methods	Fewer built-in methods

Q8. Which library would you prefer for plotting: Seaborn or Matplotlib? [2 marks]

Both are valuable but serve different purposes:

-Matplotlib: Preferred for low-level customization, fine-grained control, and basic plots. Best when precise layout control is needed.

-Seaborn: Preferred for statistical visualizations (heatmaps, violin plots, pair plots). It has better default aesthetics and requires less code for complex charts.

Recommendation: Use Seaborn for exploratory data analysis and Matplotlib when deep customization is required.

Q9. Difference between a Series and a DataFrame in Pandas? [2 marks]

-Series: A 1-dimensional labeled array capable of holding any data type. It has an index and a single column of data.

-DataFrame: A 2-dimensional labeled data structure with columns of potentially different types. Think of it as a table or spreadsheet.

```
s = pd.Series([1, 2, 3], index=['a','b','c'])
# Series
df = pd.DataFrame({'col1': [1,2], 'col2': [3,4]})      # DataFrame
```

Q10. How would you find duplicate values in a dataset in Python? [2 marks]

Using Pandas:

```
# Find duplicate rows
df.duplicated()              # returns boolean Series
df[df.duplicated()]          # shows duplicate rows
df['col'].duplicated()       # duplicates in one column
df.drop_duplicates()         # removes duplicate rows
```

GROUP B — Short Answer Questions (5 × 8 = 40 Marks)

Q1. Different loop control statements in Python with examples. [5 marks]

Python provides three loop control statements:

1. break — Terminates the loop immediately.

```
for i in range(10):
    if i == 5:
        break      # exits when i = 5
    print(i)       # prints 0 1 2 3 4
```

2. continue — Skips the current iteration and jumps to the next.

```
for i in range(6):
    if i == 3:
        continue   # skips when i = 3
    print(i)       # prints 0 1 2 4 5
```

3. pass — Does nothing; used as a placeholder.

```
for i in range(5):
    if i == 2:
        pass       # no action, loop continues
    print(i)       # prints 0 1 2 3 4
```

Q2. Types of Modules in Python. Explain any two in detail. [5 marks]

Types of Python Modules:

- Built-in Modules — come with Python installation
- User-defined Modules — created by the programmer
- Third-party Modules — installed via pip (e.g., NumPy, Pandas)

1. Built-in Module — math:

Provides mathematical functions like sqrt(), ceil(), floor(), pow(), log().

```
import math
print(math.sqrt(16))   # 4.0
print(math.pi)         # 3.14159...
```

2. User-defined Module:

Any .py file created by the programmer can be imported as a module using the import statement.

```
# mymodule.py
def greet(name):
    return f'Hello, {name}!'
# main.py
import mymodule
print(mymodule.greet('Erames'))    # Hello, Erames!
```

Q3. How many ways can NumPy arrays be created? Perform slicing and reshaping. [5 marks]

NumPy arrays can be created in multiple ways:-np.array() — from Python list/tuple

-np.zeros() / np.ones() / np.full() — initialized arrays

-np.arange() — like range but returns ndarray

-np.linspace() — evenly spaced values

-np.random.rand() / np.random.randint() — random arrays

-np.eye() — identity matrix

Slicing:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4])  # [20 30 40]  — elements at index 1,2,3
print(arr[::2])  # [10 30 50]  — every other element
```

Reshaping:

```
arr2d = np.arange(12).reshape(3, 4)  # 1D to 3x4 matrix
print(arr2d)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]
flat = arr2d.reshape(-1)  # flatten back to 1D
```

Q4. Handle exception using try-except block. Explain user-defined exception. [5 marks]

Exception handling prevents program crash when an error occurs at runtime.

try-except syntax:

```
try:
    num = int(input('Enter number: '))
    result = 100 / num
    print('Result:', result)
except ValueError:
    print('Invalid input! Please enter an integer.')
except ZeroDivisionError:
    print('Cannot divide by zero!')
else:
    print('No exception occurred.')
finally:
    print('Execution completed.')  # always runs
```

User-Defined Exception:

Created by inheriting from the Exception class.

```
class AgeError(Exception):
    def __init__(self, age):
        self.age = age
        super().__init__(f'Invalid age: {age}. Must be >= 0.')
def validate_age(age):
    if age < 0:
        raise AgeError(age)
    print('Age is valid:', age)
try:
    validate_age(-5)
except AgeError as e:
    print(e)   # Invalid age: -5. Must be >= 0.
```

Q5. List different operators in Python in order of their precedence. [5 marks]

Python operator precedence (highest to lowest):

Precedence	Operator	Description
1 (Highest)	( )	Parentheses / Grouping
2	**	Exponentiation (right to left)
3	+x, -x, ~x	Unary plus, minus, bitwise NOT
4	*, /, //, %	Multiplication, Division, Floor Div, Modulo
5	+, -	Addition, Subtraction
6	<<, >>	Bitwise Shift Operators
7	&	Bitwise AND
8	^	Bitwise XOR
9	|	Bitwise OR
10	==, !=, >, <, >=, <=, is, in	Comparison / Membership
11	not	Logical NOT
12	and	Logical AND
13 (Lowest)	or	Logical OR

Q6. How do you overload unary operators in Python? [5 marks]

Operator overloading allows custom classes to define behavior for built-in operators by implementing special (dunder) methods.

Unary operator special methods:

- __neg__(self) — for -object
- __pos__(self) — for +object
- __abs__(self) — for abs(object)
- __invert__(self) — for ~object

Example:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __neg__(self):              # overload - unary
        return Vector(-self.x, -self.y)
    def __abs__(self):              # overload abs()
        return (self.x**2 + self.y**2) ** 0.5
    def __repr__(self):
        return f'Vector({self.x}, {self.y})'
v = Vector(3, -4)
print(-v)       # Vector(-3, 4)
print(abs(v))   # 5.0
```

Q7. Features of inheritance. How to prevent method overriding in Python? [5 marks]

Inheritance allows a child class to reuse, extend, or override parent class methods/attributes.

Key Features:

-Code Reusability — child inherits parent methods without rewriting

-Method Overriding — child can redefine parent methods

-Multiple Inheritance — a class can inherit from multiple parents

-Multilevel Inheritance — chain of inheritance (A → B → C)

-Polymorphism — same method name behaves differently in child classes

Preventing Method Overriding:

Python has no built-in 'final' keyword. Prevention is achieved by:

-Convention: Prefix method name with double underscore (__) to trigger name mangling.

```
class Parent:
    def __secure_method(self):    # becomes _Parent__secure_method
        return 'Cannot be overridden easily'
```

-Custom enforcement: Check type in the method and raise an error.

```
class Parent:
    def important(self):
        if type(self) != Parent:
            raise TypeError('Override not allowed!')
        return 'Original behavior'
```

Q8. Plot height vs weight for persons aged 8–16. [5 marks]

The question requires plotting weight on x-axis and height on y-axis with a green line.

Python Code:

```
import matplotlib.pyplot as plt
height = [121.9, 124.5, 129.5, 134.6, 139.7, 147.3, 152.4, 157.5, 162.6]
weight = [19.7, 21.3, 23.5, 25.9, 28.5, 32.1, 35.7, 39.6, 43.2]
plt.figure(figsize=(8, 5))
plt.plot(weight, height, color='green', marker='o', linewidth=2)
plt.xlabel('Weight in kg', fontsize=12)
plt.ylabel('Height in cm', fontsize=12)
plt.title('Average Height vs Weight (Ages 8-16)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Output: A green line plot with x-axis showing weight (19.7 to 43.2 kg) and y-axis showing height (121.9 to 162.6 cm), showing a positive correlation between age-related height and weight growth.

Questions (10 × 4 = 40 Marks)

Q1. Dictionary operations + Program for name:mobile search [4+6] [10 marks]

Part A: Four Dictionary Operations [4 marks]

1. Creating a Dictionary:

A dictionary stores key-value pairs. Keys must be unique and immutable.

```
student = {'name': 'Ram', 'age': 21, 'city': 'Kathmandu'}
```

2. Accessing Elements:

Use key name directly or .get() method (safer, no KeyError).

```
print(student['name'])        # Ram
print(student.get('age'))     # 21
print(student.get('grade', 'N/A'))  # N/A (default if missing)
```

3. Updating / Adding Entries:

```
student['age'] = 22           # update existing key
student['grade'] = 'A'        # add new key-value pair
student.update({'city': 'Pokhara', 'email': 'ram@mail.com'})
```

4. Deleting Entries:

```
del student['email']          # delete specific key
removed = student.pop('grade')  # remove and return value
student.clear()               # empty the entire dictionary
```

Part B: Name:Mobile Dictionary Search Program [6 marks]

```
# Program to build a name:mobile dictionary and search
def build_and_search():
    contacts = {}
    n = int(input('How many contacts to add? '))
    # Build dictionary
    for i in range(n):
        name   = input(f'Enter name {i+1}: ').strip()
        mobile = input(f'Enter mobile for {name}: ').strip()
        contacts[name] = mobile
    print('\n--- Contact Book ---')
    for name, mob in contacts.items():
        print(f'{name}: {mob}')
    # Search
    search_name = input('\nEnter name to search: ').strip()
    if search_name in contacts:
        print(f'Mobile of {search_name}: {contacts[search_name]}')
    else:
        print(f'{search_name} not found in contacts.')
build_and_search()
```

Sample Output:

```
How many contacts to add? 3
Enter name 1: Ram     Enter mobile for Ram: 9841000001
Enter name 2: Sita    Enter mobile for Sita: 9841000002
Enter name 3: Hari    Enter mobile for Hari: 9841000003
Enter name to search: Sita
Mobile of Sita: 98410000
```

Q2. Define file. How to create, read, and write data files in Python? [10 marks]

A file is a named location on disk used to store data permanently. Unlike variables (which are temporary), files retain data even after the program terminates.

Mode	Description
'r'	Read (default). Error if file doesn't exist.
'w'	Write. Creates new file or overwrites existing.
'a'	Append. Creates new or adds to end of existing.
'r+'	Read and write. File must exist.
'x'	Exclusive creation. Error if file exists.
'b'	Binary mode (combine: 'rb', 'wb').

1. Creating and Writing a File:

```
# Using with statement (auto-closes file)
with open('students.txt', 'w') as f:
    f.write('Ram Bahadur\n')
    f.write('Sita Kumari\n')
    f.write('Hari Prasad\n')
print('File created and written successfully.')
```

2. Reading a File:

```
# Method 1: read() — reads entire file as string
with open('students.txt', 'r') as f:
    content = f.read()
    print(content)
# Method 2: readline() — reads one line at a time
with open('students.txt', 'r') as f:
    line = f.readline()
    while line:
        print(line.strip())
        line = f.readline()
# Method 3: readlines() — returns list of all lines
with open('students.txt', 'r') as f:
    lines = f.readlines()
    for line in lines:
        print(line.strip())
```

3. Appending to a File:

```
with open('students.txt', 'a') as f:
    f.write('Gita Devi\n')  # adds to end without erasing
```

4. Writing Structured Data (using writelines):

```
students = ['Ram\n', 'Sita\n', 'Hari\n']
with open('list.txt', 'w') as f:
    f.writelines(students)
```

Q3. Explain Pandas and perform DataFrame operations. [10 marks]

Pandas is an open-source Python library built on top of NumPy, providing high-performance, easy-to-use data structures (Series and DataFrame) and data analysis tools. It is the most widely used library for data manipulation in Python.

1. Create a Pandas DataFrame:

```
import pandas as pd
# From dictionary
data = {
    'Name':  ['Ram', 'Sita', 'Hari', 'Gita'],
    'Score': [85, 92, 78, 95],
    'Grade': ['B', 'A', 'C', 'A']
}
df = pd.DataFrame(data)
print(df)
```

2. Adding an Index to a DataFrame:

```
# Custom string index
df.index = ['s001', 's002', 's003', 's004']
print(df)
```

3. Adding Rows to a DataFrame:

```
# Using pd.concat (recommended in newer Pandas)
new_row = pd.DataFrame([['Mohan', 88, 'B']],
columns=['Name','Score','Grade'])
df = pd.concat([df, new_row], ignore_index=True)
print(df)
```

4. Resetting the Index of a DataFrame:

```
# After filtering/sorting, index may be non-sequential
df_filtered = df[df['Score'] > 80]
df_reset = df_filtered.reset_index(drop=True)  # drop=True removes old index
print(df_reset)
```

5. Deleting Rows or Columns:

```
# Drop column
df = df.drop(columns=['Grade'])
# Drop row by index label
df = df.drop(index=2)         # drop row at index 2
# Drop rows by condition
df = df[df['Score'] >= 80]    # keep only scores >= 80
```

Q4. Create ecommerce DataFrame, descriptive statistics, and student performance visualization. [10 marks]

Complete Program:

```
import pandas as pd
import matplotlib.pyplot as plt
# --- Part 1: Ecommerce DataFrame & Descriptive Statistics ---
sales = {
    'InvoiceNo':   [1001, 1002, 1903, 1004, 1085, 1006, 1007],
    'ProductName':
['LCD','AC','Deodrant','Jeans','Books','Shoes','Jacket'],
    'Quantity':    [2, 1, 2, 1, 2, 1, 1],
    'Price':       [65000, 55000, 500, 3000, 958, 3000, 2200]
}
df = pd.DataFrame(sales)
df['TotalValue'] = df['Quantity'] * df['Price']   # derived column
print('=== Descriptive Statistics ===')
print(df[['Quantity','Price','TotalValue']].describe())
# Mean, Median, Mode, Quartile, Variance
print('Mean Price  :', df['Price'].mean())
print('Median Price:', df['Price'].median())
print('Mode Price  :', df['Price'].mode()[0])
print('Q1  Price   :', df['Price'].quantile(0.25))
print('Q3  Price   :', df['Price'].quantile(0.75))
print('Variance    :', df['Price'].var())
print('Std Dev     :', df['Price'].std())
```

Expected Output:

```
Mean Price  : 18522.57
Median Price: 3000.0
Mode Price  : 3000
Q1  Price   : 1579.0
Q3  Price   : 60000.0
Variance    : 616,073,095.24
```

```
# --- Part 2: Student Performance Bar Chart ---
school_data = {
    'Subject': ['Math','Science','English','Nepali','Computer'],
    'Marks':   [78, 85, 72, 88, 92]
}
stu_df = pd.DataFrame(school_data)
plt.figure(figsize=(9, 5))
colors = ['#3498db','#2ecc71','#e74c3c','#f39c12','#9b59b6']
bars = plt.bar(stu_df['Subject'], stu_df['Marks'],
color=colors, edgecolor='black')
# Add value labels on bars
for bar in bars:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             str(bar.get_height()), ha='center', va='bottom',
fontsize=11)
plt.xlabel('Subject', fontsize=12)
plt.ylabel('Marks', fontsize=12)
plt.title('Student Performance by Subject', fontsize=14,
fontweight='bold')
plt.ylim(0, 100)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Q1. What is the difference between a keyword and an identifier in Python? **[2 Marks]**

Model Answer:

Keywords are reserved words predefined by Python that have special meaning and cannot be used as variable names (e.g., if, else, while, for, def, class, return, import, True, False, None).

Identifiers are user-defined names used to identify variables, functions, classes, modules, or other objects. They must start with a letter or underscore, and can contain letters, digits, and underscores.

Q2. List any four escape sequences in Python with their purpose. **[2 Marks]**

Model Answer:

\n → Newline (moves cursor to next line)
\t → Horizontal Tab (adds tab spacing)
\\ → Backslash (prints a literal backslash)
\' → Single Quote (prints a literal single quote)
\" → Double Quote (prints a literal double quote)
\r → Carriage Return

Q3. What is operator precedence? Give an example showing precedence between '+' and '*'. **[2 Marks]**

Model Answer:

Operator precedence determines the order in which operators are evaluated in an expression. When multiple operators appear, higher-precedence operators are evaluated first.

The '*' (multiplication) operator has higher precedence than '+' (addition).

```
result = 3 + 4 * 2
# Evaluated as: 3 + (4 * 2) = 3 + 8 = 11
print(result)  # Output: 11
```

Q4. Differentiate between mutable and immutable data types in Python with examples. **[2 Marks]**

Model Answer:

Mutable objects can be changed after creation.
Immutable objects cannot be changed after creation.

```
# Mutable — List
lst = [1, 2, 3]
lst[0] = 99       # Allowed: list is mutable
# Immutable — Tuple
tup = (1, 2, 3)
# tup[0] = 99     # TypeError: tuple does not support item assignment
# Other Immutables: int, float, str, bool, frozenset
# Other Mutables: list, dict, set
```

Q5. What is the purpose of the `global` keyword in Python? **[2 Marks]**

Model Answer:

The 'global' keyword is used inside a function to declare that a variable refers to the global scope (module-level) rather than creating a new local variable.

```
count = 0
def increment():
    global count   # without this, count would be local
    count += 1
increment()
print(count)  # Output: 1
```

Q6. What is list comprehension? Write its general syntax. **[2 Marks]**

Model Answer:

List comprehension is a concise way to create lists in Python. It allows generating a new list by applying an expression to each item in an iterable, optionally filtered by a condition.

```
# General Syntax:
[expression for item in iterable if condition]
# Example:
squares = [x**2 for x in range(1, 6)]
# Output: [1, 4, 9, 16, 25]
evens = [x for x in range(10) if x % 2 == 0]
# Output: [0, 2, 4, 6, 8]
```

Q7. Define enumeration in Python. Write a short code snippet to create an enum. **[2 Marks]**

Model Answer:

Enumeration (Enum) is a set of symbolic names (members) bound to unique, constant values. Python provides the 'enum' module to create enumerations. Enums are used to represent a fixed set of related constants.

```
from enum import Enum
class Color(Enum):
    RED   = 1
    GREEN = 2
    BLUE  = 3
print(Color.RED)       # Output: Color.RED
print(Color.RED.value)  # Output: 1
```

Q8. What is method overloading in Python? How is it achieved? **[2 Marks]**

Model Answer:

Method overloading means defining multiple methods with the same name but different parameters. Python does not natively support method overloading — only the last defined version is used. It is achieved using default arguments or *args.

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c
calc = Calculator()
print(calc.add(5))       # Output: 5
print(calc.add(5, 3))    # Output: 8
print(calc.add(5, 3, 2)) # Output: 10
```

Q9. What is a NumPy universal function (ufunc)? Give two examples. **[2 Marks]**

Model Answer:

A Universal Function (ufunc) in NumPy is a function that operates element-wise on arrays, supporting broadcasting, type casting, and vectorised operations. They are significantly faster than equivalent Python loops.

```
import numpy as np
arr = np.array([0, np.pi/2, np.pi])
# Example 1: np.sin() — element-wise sine
print(np.sin(arr))  # [0.  1.  0.]
# Example 2: np.sqrt() — element-wise square root
print(np.sqrt(np.array([4, 9, 16])))  # [2. 3. 4.]
```

Q10. What is hierarchical indexing in Pandas? Where is it used? **[2 Marks]**

Model Answer:

Hierarchical Indexing (MultiIndex) allows multiple index levels on an axis in Pandas. It enables representing higher-dimensional data in a lower-dimensional form (like a 2D DataFrame representing 3D data).

```
import pandas as pd
arrays = [['A','A','B', 'B'], [1, 2, 1, 2]]
index  = pd.MultiIndex.from_arrays(arrays,
names=('Group', 'Sub'))
df = pd.DataFrame({'Value': [10, 20, 30, 40]},
index=index)
print(df)
# Used in: pivot tables, grouped data, time series with
date + hour
```

Q11. What are literals in Python? Give examples of integer, float, string, and boolean literals. **[2 Marks]**

Model Answer:

Literals are fixed/constant values directly written in code that do not change during program execution. Python supports several types of literals:

```
age     = 25        # Integer literal
price   = 19.99     # Float literal
name    = 'Nepal'   # String literal
flag    = True      # Boolean literal
nothing = None      # None literal
hex_val = 0xFF      # Hexadecimal integer literal →
255
```

Q12. Write the syntax and purpose of the break, continue, and pass jump statements. **[2 Marks]**

Model Answer:

```
# break — exits the loop immediately
for i in range(10):
    if i == 5:
        break
    print(i)  # prints 0 to 4
# continue — skips current iteration, moves to next
for i in range(5):
    if i == 2:
        continue
    print(i)  # prints 0,1,3,4
# pass — does nothing; used as placeholder
def future_function():
    pass      # no error, but empty body
```

Q13. What is the difference between == and is operators in Python? **[2 Marks]**

Model Answer:

== (Equality operator): Compares the values of two objects.

is (Identity operator): Checks if two variables point to the same object in memory (same id).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)  # True  — same value
print(a is b)  # False — different objects in memory
print(a is c)  # True  — c points to same object as a
```

Q14. Define recursive function. Write a Python function to compute factorial recursively. **[2 Marks]**

Model Answer:

A recursive function is a function that calls itself to solve a smaller sub-problem of the same type. It must have a base case to prevent infinite recursion.

```
def factorial(n):
    if n == 0 or n == 1:  # Base case
        return 1
    return n * factorial(n - 1)  # Recursive call
print(factorial(5))  # Output: 120
# 5 * 4 * 3 * 2 * 1 = 120
```

Q15. What are functional tools in Python? Name and briefly explain map(), filter(), and reduce(). **[2 Marks]**

Model Answer:

Functional tools apply a function to iterables without explicit loops, promoting a functional programming style.

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
# map() — applies function to every element
squares = list(map(lambda x: x**2, nums))  #
[1,4,9,16,25]
# filter() — keeps elements where function returns True
evens   = list(filter(lambda x: x%2==0, nums))  # [2,4]
# reduce() — reduces list to a single value
total= reduce(lambda x, y: x + y, nums)# 15
```

Q16. What is the difference between a Tuple and a Set in Python? **[2 Marks]**

Tuple	Set
Ordered (maintains insertion order)	Unordered (no guaranteed order)
Allows duplicate values	No duplicate values allowed
Immutable (cannot be modified)	Mutable (can add/remove elements)
Defined with ( ): (1, 2, 3)	Defined with { }: {1, 2, 3}
Supports indexing/slicing	Does not support indexing

Q17. What is file handling in Python? List the different file opening modes. **[2 Marks]**

File handling allows a program to create, read, write, and manipulate files on the file system using built-in functions like open().

```
# Syntax: open(filename, mode)
'r'  → Read only (default). File must exist.
'w'  → Write only. Creates new or overwrites existing.
'a'  → Append. Adds to end of file without overwriting.
'x'  → Exclusive creation. Fails if file exists.
'r+' → Read and Write.
'b'  → Binary mode (combine: 'rb', 'wb')
with open('data.txt', 'r') as f:
    content = f.read()
```

Q18. What is the difference between single and multiple inheritance in Python? **[2 Marks]**

Model Answer:

Single Inheritance: A child class inherits from only ONE parent class.

Multiple Inheritance: A child class inherits from MORE THAN ONE parent class simultaneously.

```
# Single Inheritance
class Animal:
    def breathe(self): print('Breathing')
class Dog(Animal):   # inherits from one class
    def bark(self): print('Woof!')
# Multiple Inheritance
class Flyable:
    def fly(self): print('Flying')
class Swimmable:
    def swim(self): print('Swimming')
class Duck(Flyable, Swimmable):  # inherits from two
classes
    Pass
```

Q19. What is a Pandas Series? How does it differ from a NumPy array? **[2 Marks]**

Model Answer:

A Pandas Series is a one-dimensional labeled array capable of holding any data type. Unlike NumPy arrays, each element has an associated index (label).

```
import pandas as pd
import numpy as np
np_arr = np.array([10, 20, 30])    # NumPy array
pd_ser = pd.Series([10, 20, 30],   # Pandas Series
index=['a','b','c'])
print(pd_ser['b'])  # Access by label: 20
print(np_arr[1])    # Access only by integer
position: 20
```

Pandas Series	NumPy Array
Has labeled index (default 0,1,2...)	Only integer positional index
Supports heterogeneous dtypes	Homogeneous dtype (same type)
Part of Pandas library	Part of NumPy library
Richer data alignment features	Faster for numerical computation

Q20. What is random number generation in NumPy? Give two functions used for it.

NumPy provides the numpy.random module for generating random numbers, arrays, and distributions. It is widely used in simulations, machine learning, and statistical analysis.

```
import numpy as np
# Function 1: np.random.rand() — uniform
distribution [0.0, 1.0)
arr1 = np.random.rand(3, 3)  # 3x3 array of
random floats
# Function 2: np.random.randint() — random
integers in a range
arr2 = np.random.randint(1, 100, size=(4,))  # [45,
7, 83, 22]
# Other common functions:
# np.random.randn()  — standard normal
distribution
# np.random.choice() — random sample from
array
# np.random.seed()   — for reproducible results
```

Q1. Conditional Statements: if, elif, else & Ternary Operator [5 Marks]

▸ **Answer:**

Conditional statements allow a program to make decisions and execute different code blocks based on conditions. Python uses indentation to define code blocks.

1.1 Syntax

```
if condition1:
    # block 1
elif condition2:
    # block 2
else:
    # default block
```

1.2 Example — Grade Classification

```
marks = 75
if marks >= 90:
    print('Grade: A')
elif marks >= 80:
    print('Grade: B')
elif marks >= 60:
    print('Grade: C')
else:
    print('Grade: F')
# Output: Grade: C
```

1.3 Ternary Operator

The ternary (conditional) operator provides a one-line shorthand for simple if-else:

```
# Syntax: value_if_true if condition else value_if_false
age = 20
status = 'Adult' if age >= 18 else 'Minor'
print(status)  # Output: Adult
# Another example
x, y = 10, 20
maximum = x if x > y else y
print(f'Max: {maximum}')  # Output: Max: 20
```

Key Points:

-Python does not have switch-case (Python 3.10+ has match-case).

-Conditions can be combined using 'and', 'or', 'not'.

-Indentation (4 spaces or 1 tab) is mandatory

Q2. Types of Loops — While Loop: Fibonacci Series [5 Marks]

▸ **Answer:**

Python has two primary loop constructs:

Loop	Use Case	Syntax
for loop	Iterate over sequences (list, range, string)	for item in sequence:
while loop	Repeat while a condition is True	while condition:

2.1 for Loop — Example

```
# Iterate over a list
fruits = ['apple', 'mango', 'banana']
for fruit in fruits:
    print(fruit)
# Using range()
for i in range(1, 6):  # 1 to 5
    print(i, end=' ')  # 1 2 3 4 5
```

2.2 while Loop — Fibonacci Series up to n Terms

```
def fibonacci(n):
    a, b = 0, 1
    count = 0
    print(f'Fibonacci Series ({n} terms):')
    while count < n:
        print(a, end=' ')
        a, b = b, a + b
        count += 1
    print()
fibonacci(10)
# Output: 0 1 1 2 3 5 8 13 21 34
```

Q3. Functions: *args, **kwargs & Multiple Return Values [5 Marks]

▸ **Answer:**

A function is a reusable block of code that performs a specific task. Functions improve modularity and reduce repetition.

3.1 Defining a Function

```
def greet(name):
    return f'Hello, {name}!'
print(greet('Alice'))#Output: Hello, Alice!
```

3.2 *args — Variable Positional Arguments

*args collects any number of positional arguments into a tuple:

```
def total(*args):
    return sum(args)
print(total(1, 2, 3))      # 6
print(total(10, 20, 30, 40)) # 100
def show_args(*args):
    for i, val in enumerate(args):
        print(f'  arg[{i}] = {val}')
show_args('Python', 3.14, True)
```

3.3 **kwargs — Variable Keyword Arguments

**kwargs collects keyword arguments into a dictionary:

```
def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f'  {key}: {value}')
display_info(name='Alice', age=25, city='Kathmandu')
```

3.4 Returning Multiple Values

Python functions can return multiple values as a tuple:

```
def min_max_avg(numbers):
    return min(numbers), max(numbers), sum(numbers)/len(numbers)
data = [4, 7, 2, 9, 1, 8]
minimum, maximum, average = min_max_avg(data)
print(f'Min: {minimum}, Max: {maximum}, Avg: {average:.2f}')
# Output: Min: 1, Max: 9, Avg: 5.17
```

Q4. String Operations: Slicing, Concatenation, f-strings & Methods [5 Marks]

▸ **Answer:**

Strings in Python are immutable sequences of characters. Python provides rich built-in string manipulation features.

4.1 String Slicing

```
# Syntax: string[start:stop:step]
text = 'Python Programming'
print(text[0:6])     # 'Python'
print(text[7:])      # 'Programming'
print(text[::-1])    # 'gnimmargorP nohtyP'  (reversed)
print(text[::2])     # 'Pto rgamn'
print(text[-11:])    # 'Programming'
```

4.2 String Concatenation

```
first = 'Hello'    s
econd = 'World'
# Using + operator
result = first + ', ' + second + '!'
print(result)  # Hello, World!
# Using join() — more efficient for many strings
words = ['Python', 'is', 'powerful']
print(' '.join(words))  # Python is powerful
```

4.3 Formatted Strings (f-strings) [Python 3.6+]

```
name = 'Alice'
score = 95.678
print(f'Student: {name}')
print(f'Score: {score:.2f}')   # 2 decimal places
print(f'Result: {score > 50}')  # Expression inside {}
print(f'Name upper: {name.upper()}')
# Multi-line f-string
info = (f'Name: {name}\n'
        f'Score: {score:.1f}')
print(info)
```

4.4 String Methods

Method	Description	Example → Output
upper() / lower()	Convert case	"hello".upper() → 'HELLO'
strip()	Remove whitespace	" hi ".strip() → 'hi'
split()	Split into list	"a,b,c".split(',') → ['a','b','c']
replace()	Replace substring	"cats".replace('c','b') → 'bats'
find()	Find index of substring	"hello".find('ll') → 2
startswith()	Check prefix	"Python".startswith('Py') → True
count()	Count occurrences	"banana".count('a') → 3

Q5. Compare: Lists, Tuples, Sets & Dictionaries [5 Marks]

▸ **Answer:**

Feature	List	Tuple	Set	Dictionary
Syntax	[ ]	( )	{ }	{ key: val }
Mutable	Yes	No	Yes	Yes
Ordered	Yes (3.7+)	Yes	No	Yes (3.7+)
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	Yes	Yes	No	By key
Hashable	No	Yes	Elements yes	Keys yes

Examples:

```
# LIST — mutable, ordered, allows duplicates
fruits = ['apple', 'mango', 'apple', 'banana']
fruits.append('cherry')    # ['apple','mango','apple','banana','cherry']
fruits[1] = 'grape'        # Modify by index
# TUPLE — immutable, ordered
coordinates = (27.7172, 85.3240)  # lat, lon of Kathmandu
# coordinates[0] = 28  # Error! Tuples are immutable
lat, lon = coordinates           # Unpacking
# SET — unique elements, unordered
ids = {101, 102, 103, 101, 102}
print(ids)          # {101, 102, 103} (duplicates removed)
ids.add(104)        # Add element
print(105 in ids)   # Membership test: False
# DICTIONARY — key-value pairs
student = {'name': 'Ram', 'age': 20, 'grade': 'A'}
print(student['name'])          # Ram
student['marks'] = 85           # Add new key
for k, v in student.items():
    print(f'{k}: {v}')
```

Q6. Exception Handling: try, except, else, finally, raise [5 Marks]

▸ **Answer:**

Exception handling allows a program to gracefully handle runtime errors (exceptions) instead of crashing.

6.1 Structure

```
try:
    # Code that might raise an exception
except ExceptionType as e:
    # Handle specific exception
else:
    # Runs if NO exception occurred
finally:
    # ALWAYS runs (cleanup code)
```

6.2 Comprehensive Example

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print('Error: Cannot divide by zero!')
        return None
    except TypeError as e:
        print(f'TypeError: {e}')
        return None
    else:
        print(f'Division successful: {result:.4f}')
        return result
    finally:
        print('--- Operation Complete ---')
safe_divide(10, 2)    # Division successful: 5.0000
safe_divide(10, 0)    # Error: Cannot divide by zero!
```

6.3 raise — Custom Exceptions

```
class AgeError(Exception):
    pass
def set_age(age):
    if age < 0 or age > 150:
        raise AgeError(f'Invalid age: {age}')
    print(f'Age set to {age}')
try:
    set_age(-5)
except AgeError as e:
    print(f'Caught: {e}')   # Caught: Invalid age: -5
```

Write a Python program using Matplotlib to display a bar chart showing the number of students in different classes. Use different colors for each bar and label the chart properly.

Classes	1	2	3	4	5	6	7	8	9	10
No. of Students	42	44	45	50	49	28	51	57	41	

```
import matplotlib.pyplot as plt

# Data
classes = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
students = [42, 44, 45, 50, 49, 28, 51, 57, 41, 40]

# Different colors for bars
colors = ['red', 'blue', 'green', 'orange', 'purple',
          'cyan', 'pink', 'yellow', 'brown', 'gray']

# Create bar chart
plt.bar(classes, students, color=colors)

# Labels and title
plt.xlabel("Classes")
plt.ylabel("Number of Students")
plt.title("Number of Students in Different Classes")

# Show class numbers on x-axis
plt.xticks(classes)

# Display chart
plt.show()
```

Q7. Class Methods vs Static Methods vs Instance Methods [5 Marks]

▸ **Answer:**

Type	Decorator	First Parameter	Access
Instance Method	None	self	Instance attrs + class attrs
Class Method	@classmethod	cls	Class attrs only
Static Method	@staticmethod	None	No self or cls

Example:

```
class Employee:
    company = 'TechCorp'        # Class attribute
    employee_count = 0
    def __init__(self, name, salary):
        self.name = name        # Instance attribute
        self.salary = salary
        Employee.employee_count += 1
    # Instance method — works on individual object
    def get_details(self):
        return f'{self.name} earns Rs.{self.salary}'
    # Class method — works on class itself
    @classmethod
    def get_count(cls):
        return f'Total employees: {cls.employee_count}'
    # Static method — utility, no class/instance needed
    @staticmethod
    def is_valid_salary(salary):
        return salary > 0
e1 = Employee('Alice', 50000)
e2 = Employee('Bob', 60000)
print(e1.get_details())           # Instance method
print(Employee.get_count())       # Class method
print(Employee.is_valid_salary(-100)) # Static method: False
```

Q8. Operator Overloading — Vector Class [5 Marks]

▸ **Answer:**

Operator overloading allows user-defined classes to define behavior for built-in operators like +, -, *, == etc. This is done by implementing special (dunder) methods.

Common Dunder Methods:

Operator	Dunder Method
+	__add__(self, other)
-	__sub__(self, other)
*	__mul__(self, other)
==	__eq__(self, other)
str()	__str__(self)
len()	__len__(self)

Vector Class with + and * Overloading:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f'Vector({self.x}, {self.y})'
    def __add__(self, other):
        # Add two vectors component-wise
        return Vector(self.x + other.x, self.y + other.y)
    def __mul__(self, scalar):
        # Scalar multiplication
        return Vector(self.x * scalar, self.y * scalar)
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def magnitude(self):
        return (self.x**2 + self.y**2) ** 0.5
v1 = Vector(3, 4)
v2 = Vector(1, 2)
print(v1 + v2)      # Vector(4, 6)
print(v1 * 3)       # Vector(9, 12)
print(v1 == v2)     # False
print(v1.magnitude()) # 5.0
```

Q9. Data Types & Type Conversion (Implicit & Explicit) [5 Marks]

▸ **Answer:**

9.1 Python Built-in Data Types

Category	Type	Example
Numeric	int	x = 42
Numeric	float	y = 3.14
Numeric	complex	z = 2+3j
Text	str	s = 'hello'
Boolean	bool	b = True
Sequence	list, tuple, range	[1,2,3]
Mapping	dict	{'a': 1}
Set	set, frozenset	{1, 2, 3}

9.2 Implicit Type Conversion (Coercion)

Python automatically converts to a 'higher' type to avoid data loss:

```
a = 10        # int
b = 3.14      # float
c = a + b     # Python converts int → float automatically
print(c)      # 13.14
print(type(c)) # <class 'float'>
# int + bool → int (True=1, False=0)
print(10 + True)   # 11
print(10 + False)  # 10
```

9.3 Explicit Type Conversion (Casting)

```
# int() — convert to integer
print(int(3.99))      # 3  (truncates, not rounds)
print(int('42'))      # 42
print(int(True))      # 1
# float() — convert to float
print(float('3.14'))  # 3.14
print(float(10))      # 10.0
# str() — convert to string
print(str(100))       # '100'
print(str(3.14))      # '3.14'
# bool() — convert to boolean (0, '', [], None → False)
print(bool(0))        # False
print(bool(''))       # False
print(bool([]))       # False
print(bool('hello'))  # True
print(bool(42))       # True
# Practical: user input is always string
age_str = '25'
age_int = int(age_str)
print(f'Next year: {age_int + 1}')
# 26
```

Q10. User-Defined Functions: Default Args, Keyword Args & Return Objects [5 Marks]

▸ **Answer:**

10.1 Default Arguments

Default values are used when the caller doesn't provide that argument:

```
def connect(host, port=3306, timeout=30):
    return f'Connecting to {host}:{port} (timeout={timeout}s)'
print(connect('localhost'))            # port=3306, timeout=30
print(connect('db.example.com', 5432)) # port=5432, timeout=30
print(connect('server', 8080, 60))     # All specified
```

10.2 Keyword Arguments

Caller specifies argument names explicitly — order doesn't matter:

```
def register(name, age, city='Unknown'):
    print(f'{name}, {age} years, from {city}')
register(age=22, name='Alice', city='Pokhara')
register('Bob', city='Lalitpur', age=25)
```

10.3 Functions Returning Objects

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def __str__(self):
        return f'{self.name}: {self.marks}'
def create_student(name, *scores):
    avg = sum(scores) / len(scores)
    return Student(name, round(avg, 2))
s = create_student('Alice', 80, 90, 75, 85)
print(s)  # Alice: 82.5
# Return list of objects
def top_students(students, n=3):
    return sorted(students, key=lambda s: s.marks, reverse=True)[:n]
```

Q11. List Operations & List Comprehension [5 Marks]

▸ **Answer:**

11.1 Core List Operations

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]
nums.append(7)       # [3,1,4,1,5,9,2,6,7] — add to end
nums.insert(2, 99)   # [3,1,99,4,1,5,9,2,6,7] — insert at index 2
nums.remove(99)      # removes first occurrence of 99
popped = nums.pop()  # removes & returns last element
popped2 = nums.pop(0) # removes & returns element at index 0
nums.sort()          # ascending sort in-place
print(nums)          # [1, 1, 2, 4, 5, 6, 9]
nums.reverse()       # reverses in-place
print(nums)          # [9, 6, 5, 4, 2, 1, 1]
# Other useful methods
print(nums.count(1))  # 2 — count occurrences
print(nums.index(5))  # find index of 5
nums2 = nums.copy()   # shallow copy
nums.clear()          # empty the list
```

11.2 List Comprehension — Filter Even Numbers

```
# Traditional way
numbers = list(range(1, 21))
evens = []
for n in numbers:
    if n % 2 == 0:
        evens.append(n)
# Pythonic way — List Comprehension
evens = [n for n in range(1, 21) if n % 2 == 0]
print(evens)  # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
# Comprehension with transformation
squares_of_evens = [n**2 for n in range(1, 11) if n % 2 == 0]
print(squares_of_evens)  # [4, 16, 36, 64, 100]
# Nested list comprehension — flatten 2D list
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [x for row in matrix for x in row]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

Q12. Regular Expressions with the re Module [5 Marks]

▸ **Answer:**

Regular expressions (regex) are patterns used to match, search, and manipulate strings. Python's re module provides full regex support.

12.1 Common Regex Patterns

Pattern	Matches
\d	Any digit (0-9)
\w	Word character (a-z, A-Z, 0-9, _)
\s	Whitespace
.	Any character except newline
^	Start of string
$	End of string
+	One or more
*	Zero or more
?	Zero or one
{n,m}	Between n and m times

12.2 re Functions with Examples

```
import re
text = 'Call us at 98-4567-8901 or email info@example.com'
# re.match() — matches at the BEGINNING of string
result = re.match(r'Call', text)
print(result.group() if result else 'No match')  # Call
# re.search() — searches ANYWHERE in string
phone = re.search(r'\d{2}-\d{4}-\d{4}', text)
if phone:
    print(f'Phone: {phone.group()}')  # 98-4567-8901
# re.findall() — returns ALL matches as list
text2 = 'Apples cost Rs.50, Mangoes cost Rs.120, Bananas Rs.30'
prices = re.findall(r'Rs\.\d+', text2)
print(prices)  # ['Rs.50', 'Rs.120', 'Rs.30']
# re.sub() — substitute/replace matches
censored = re.sub(r'\d+', '***', 'My PIN is 1234 and OTP is 5678')
print(censored)  # My PIN is *** and OTP is ***
# Email validation
def is_valid_email(email):
    pattern = r'^[\w.-]+@[\w.-]+\.\w{2,}$'
    return bool(re.match(pattern, email))
print(is_valid_email('user@gmail.com'))  # True
print(is_valid_email('invalid-email'))   # False
```

Q13. OOP: Constructors, Access Modifiers & BankAccount Class [5 Marks]

▸ **Answer:**

13.1 Constructor — __init__

The constructor (__init__) is a special method called when an object is created. It initializes object attributes.

13.2 Access Modifiers in Python

Modifier	Syntax	Access Level
Public	name	Accessible anywhere
Protected	_name	Convention: internal use (not enforced)
Private	__name	Name-mangled: _ClassName__name

13.3 BankAccount Class

```
class BankAccount:
    bank_name = 'Nepal Bank Ltd'    # Class attribute
    def __init__(self, owner, initial_balance=0):
        self.owner = owner          # Public
        self._account_no = self.__generate_account_no()  # Protected
        self.__balance = initial_balance  # Private
        self.__transactions = []          # Private
    def __generate_account_no(self):
        import random
        return f'AC{random.randint(100000, 999999)}'
    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('Deposit amount must be positive')
        self.__balance += amount
        self.__transactions.append(f'Deposited Rs.{amount}')
        print(f'Rs.{amount} deposited. New balance: Rs.{self.__balance}')
    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError('Amount must be positive')
        if amount > self.__balance:
            raise ValueError('Insufficient funds')
        self.__balance -= amount
        self.__transactions.append(f'Withdrew Rs.{amount}')
        print(f'Rs.{amount} withdrawn. Remaining: Rs.{self.__balance}')
    def enquiry(self):
        print(f'Account Holder: {self.owner}')
        print(f'Account No:     {self._account_no}')
        print(f'Balance:        Rs.{self.__balance}')
    def statement(self):
        print('--- Transaction History ---')
        for t in self.__transactions:
            print(f'  {t}')
acc = BankAccount('Ram Prasad', 5000)
acc.deposit(2000)    # Rs.2000 deposited. New balance: Rs.7000
acc.withdraw(1500)   # Rs.1500 withdrawn. Remaining: Rs.5500
acc.enquiry()
acc.statement()
# print(acc.__balance)  # AttributeError! Private attribute
```

Q14. zip() Function & Argument Unpacking [5 Marks]

▸ **Answer:**

14.1 zip() Function

zip() combines two or more iterables element-wise into an iterator of tuples:

```
names = ['Alice', 'Bob', 'Charlie']
scores = [85, 92, 78]
grades = ['B', 'A', 'C']
# Basic zip
combined = list(zip(names, scores))
print(combined)  # [('Alice', 85), ('Bob', 92), ('Charlie', 78)]
# Zip three iterables
for name, score, grade in zip(names, scores, grades):
    print(f'{name}: {score} ({grade})')
# zip stops at shortest iterable
a = [1, 2, 3, 4, 5]
b = ['x', 'y', 'z']
print(list(zip(a, b)))  # [(1,'x'), (2,'y'), (3,'z')]
# zip to create dictionary
keys = ['name', 'age', 'city']
vals = ['Alice', 25, 'Kathmandu']
d = dict(zip(keys, vals))
print(d)  # {'name': 'Alice', 'age': 25, 'city': 'Kathmandu'}
```

14.2 Unzipping with *

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
numbers, letters = zip(*pairs)   # Unzip
print(numbers)  # (1, 2, 3)
print(letters)  # ('a', 'b', 'c')
```

14.3 Argument Unpacking in Function Calls

```
def add(a, b, c):
    return a + b + c
# * unpacks a list into positional arguments
values = [1, 2, 3]
print(add(*values))     # 6
# ** unpacks a dict into keyword arguments
params = {'a': 10, 'b': 20, 'c': 30}
print(add(**params))    # 60
# Mix positional and unpacked
extra = [2, 3]
print(add(1, *extra))   # 6
# Useful for combining dicts (Python 3.9+)
d1 = {'x': 1, 'y': 2}
d2 = {'z': 3}
merged = {**d1, **d2}
print(merged)  # {'x': 1, 'y': 2, 'z': 3}
```

Q15. Inheritance: Method Overriding, super() & MRO [5 Marks]

▸ **Answer:**

15.1 Method Overriding

A subclass provides its own implementation of a method defined in the parent class:

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f'{self.name} makes a sound'
class Dog(Animal):
    def speak(self):  # Overrides parent method
        return f'{self.name} says: Woof!'
class Cat(Animal):
    def speak(self):  # Overrides parent method
        return f'{self.name} says: Meow!'
animals = [Dog('Rex'), Cat('Whiskers'), Animal('Bird')]
for a in animals:
    print(a.speak())  # Polymorphism in action
```

15.2 super() — Calling Parent's Method

```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed
    def info(self):
        return f'{self.brand} at {self.speed} km/h'
class ElectricCar(Vehicle):
    def __init__(self, brand, speed, battery_kw):
        super().__init__(brand, speed)  # Call parent __init__
        self.battery_kw = battery_kw
    def info(self):  # Extend parent method
        base = super().info()  # Call parent info()
        return f'{base}, Battery: {self.battery_kw}kW'
car = ElectricCar('Tesla', 250, 100)
print(car.info())  # Tesla at 250 km/h, Battery: 100kW
```

15.3 Multiple Inheritance & MRO

Python resolves method lookup order using the C3 Linearization algorithm (MRO):

```
class A:
    def hello(self): return 'A'
class B(A):
    def hello(self): return 'B'
class C(A):
    def hello(self): return 'C'
class D(B, C):  # Multiple inheritance
    pass
d = D()
print(d.hello())        # 'B' — follows MRO
print(D.__mro__)        # D → B → C → A → object
print(D.mro())          # Same as list
```

MRO ensures: D → B → C → A → object (left-to-right, depth-first, but respects C3 rule).

Q16. NumPy Slicing, Indexing, Fancy Indexing & Boolean Indexing [5 Marks]

▸ **Answer:**

16.1 1D Array Slicing

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[2])      # 30 — single element
print(arr[1:5])    # [20 30 40 50] — slice
print(arr[::-1])   # [70 60 50 40 30 20 10] — reversed
print(arr[::2])    # [10 30 50 70] — every other element
```

16.2 2D Array Slicing

```
matrix = np.array([[1,  2,  3,  4],
                   [5,  6,  7,  8],
                   [9, 10, 11, 12]])
print(matrix[1, 2])     # 7 — row 1, col 2
print(matrix[0, :])     # [1 2 3 4] — entire first row
print(matrix[:, 2])     # [3 7 11] — entire column 2
print(matrix[1:, 1:3])  # [[6 7], [10 11]] — submatrix
print(matrix[:, ::2])   # columns 0, 2 → [[1,3],[5,7],[9,11]]
```

16.3 Fancy Indexing

Access multiple non-contiguous elements using integer arrays:

```
arr = np.array([10, 20, 30, 40, 50])
# Select specific indices
indices = [0, 2, 4]
print(arr[indices])   # [10 30 50]
# 2D fancy indexing
rows = np.array([0, 2])
cols = np.array([1, 3])
print(matrix[rows, cols])  # [matrix[0,1], matrix[2,3]] = [2, 12]
```

16.4 Boolean (Mask) Indexing

Filter elements using True/False conditions — returns a new array:

```
data = np.array([15, -3, 42, -8, 19, 0, 27, -11])
# Create boolean mask
mask = data > 0
print(mask)       # [ True False True False True False True False]
# Apply mask — returns only True elements
positives = data[mask]
print(positives)  # [15 42 19 27]
# One-liner
print(data[data > 10])  # [15 42 19 27]
# Combined conditions
print(data[(data > 0) & (data < 25)])  # [15 19]
# Use mask to modify values
data[data < 0] = 0
print(data)  # [15  0 42  0 19  0 27  0] — negatives zeroed
```

Summary — NumPy Indexing Types

Type	Syntax Example	Returns
Basic Slicing	arr[1:5], arr[::2]	View (shares memory)
Integer Indexing	arr[2], arr[1,3]	Scalar or new array
Fancy Indexing	arr[[0,2,4]]	Copy (new array)
Boolean Indexing	arr[arr > 0]	Copy (new array)

Pandas Practical Question (

Question: [10 Marks]

A school has stored students' marks in a CSV file named marks.csv with the following data:

Name	Math	Science	English
Ram	78	85	80
Sita	92	88	91
Hari	67	72	70
Gita	81	79	84

Write a Python program using Pandas to:

1. Read the CSV file.
2. Display the first 3 rows.
3. Add a new column named Total.
4. Find the student with the highest total marks.
5. Save the updated data into a new file called result.csv.

```
import pandas as pd
# Step 1: Read CSV file
df = pd.read_csv("marks.csv")
# Step 2: Display first 3 rows
print("First 3 Rows:")
print(df.head(3))
# Step 3: Add Total column
df["Total"] = df["Math"] + df["Science"] + df["English"]
# Step 4: Find highest scorer
topper = df.loc[df["Total"].idxmax()]
print("\nTopper Details:")
print(topper)
# Step 5: Save updated data
df.to_csv("result.csv", index=False)
print("\nUpdated file saved as result.csv")
```

```
# Importing pandas and numpy libraries
import pandas as pd
import numpy as np
# Creating a dummy DataFrame of 12 numbers randomly
# ranging from 150-180 for height
df = pd.DataFrame({'Height': [150.4, 157.6, 170, 176, 164.2, 155, 159.2, 175, 162.4, 176, 153, 170.9]})
# Printing DataFrame Before Sorting Continuous to Categories
print("Before: ")
print(df)
# A column of name 'Label' is created in DataFrame
# Categorizing Height into 3 Categories
# Short: (150,157], 150 is excluded & 157 is included
# Average: (157,169], 157 is excluded & 169 is included
# Tall: (169,180], 169 is excluded & 180 is included
df['Label'] = pd.cut(x=df['Height'], bins=[150, 157, 169, 180], labels=['Short', 'Average', 'Tall'])
# Printing DataFrame After Sorting Continuous to Categories
print("After: ")
print(df)
# Check the number of values in each bin
print("Categories: ")
print(df['Label'].value_counts())
```

Question:[10 Marks]

QN1. Create a DataFrame containing students' marks and perform the following operations:

Display the DataFrame.
Add a new column Total.
Find the average marks in Math.
Display the student with the highest total marks.

Solution

```
import pandas as pd
# Creating DataFrame
data = {
    "Name": ["Ram", "Sita", "Hari", "Gita"],
    "Math": [78, 92, 67, 81],
    "Science": [85, 88, 72, 79],
    "English": [80, 91, 70, 84]
}
df = pd.DataFrame(data)
# Display DataFrame
print("Student Data:")
print(df)
# Add Total column
df["Total"] = df["Math"] + df["Science"] + df["English"]
print("\nData with Total Marks:")
print(df)
# Average marks in Math
avg_math = df["Math"].mean()
print("\nAverage Marks in Math:", avg_math)
# Student with highest total marks
topper = df.loc[df["Total"].idxmax()]
print("\nTopper Details:")
print(topper)
```

QN2. Write a Python program using Pandas to create a DataFrame of employees and perform descriptive statistical analysis.

The DataFrame should contain the following data:

Name	Age	Salary	Experience
Asha	25	35000	2
Ram	30	42000	5
Sita	28	39000	4
Hari	35	50000	8
Gita	32	47000	6

Perform the following tasks:

Create the DataFrame.
Display the first 3 rows.
Find a mean salary.
Find maximum and minimum age.
Display descriptive statistics of the dataset using describe().

Solution

```
import pandas as pd
# Creating DataFrame
data = {
    "Name": ["Asha", "Ram", "Sita", "Hari", "Gita"],
    "Age": [25, 30, 28, 35, 32],
    "Salary": [35000, 42000, 39000, 50000, 47000],
    "Experience": [2, 5, 4, 8, 6]
}
df = pd.DataFrame(data)
# Display first 3 rows
print("First 3 Rows:")
print(df.head(3))
# Mean salary
print("\nMean Salary:")
print(df["Salary"].mean())
# Maximum and minimum age
print("\nMaximum Age:")
print(df["Age"].max())
print("\nMinimum Age:")
print(df["Age"].min())
# Descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())
```

Data preprocessing [10 Marks]

Data preprocessing is an important step in the data mining process. It refers to the cleaning, transforming, and integrating of data in order to make it ready for analysis. The goal of data preprocessing is to improve the quality of the data and to make it more suitable for the specific data mining task.

Some common steps in data preprocessing include:

Data Cleaning: This involves identifying and correcting errors or inconsistencies in the data, such as missing values, outliers, and duplicates. Various techniques can be used for data cleaning, such as imputation, removal, and transformation.

Data Integration: This involves combining data from multiple sources to create a unified dataset. Data integration can be challenging as it requires handling data with different formats, structures, and semantics. Techniques such as record linkage and data fusion can be used for data integration.

Data Transformation: This involves converting the data into a suitable format for analysis. Common techniques used in data transformation include normalization, standardization, and discretization. Normalization is used to scale the data to a common range, while standardization is used to transform the data to have zero mean and unit variance. Discretization is used to convert continuous data into discrete categories.

Data Reduction: This involves reducing the size of the dataset while preserving the important information. Data reduction can be achieved through techniques such as feature selection and feature extraction. Feature selection involves selecting a subset of relevant features from the dataset, while feature extraction involves transforming the data into a lower-dimensional space while preserving the important information.

Data Discretization: This involves dividing continuous data into discrete categories or intervals. Discretization is often used in data mining and machine learning algorithms that require categorical data. Discretization can be achieved through techniques such as equal width binning, equal frequency binning, and clustering.

Data Normalization: This involves scaling the data to a common range, such as between 0 and 1 or -1 and 1. Normalization is often used to handle data with different units and scales. Common normalization techniques include min-max normalization, z-score normalization, and decimal scaling.

Data preprocessing plays a crucial role in ensuring the quality of data and the accuracy of the analysis results. The specific steps involved in data preprocessing may vary depending on the nature of the data and the analysis goals.

By performing these steps, the data mining process becomes more efficient and the results become more accurate.

Create a file named csit.txt, write the text "Welcome to CSIT Department" into it, and then read and display the content of the file

```
# Create a file and write text into it
file = open("csit.txt", "w")
file. write("Welcome to CSIT")
file. Close ()
# Open the file for reading
file = open("csit.txt", "r")
filedata = file.read()
# Display the content
print("File Content:")
print(filedata)
# Close the file
file.close()
```

Preprocessing in Data Mining:

Data preprocessing is a data mining technique which is used to transform the raw data in a useful and efficient format.

Steps Involved in Data Preprocessing:

1. Data Cleaning:

The data can have many irrelevant and missing parts. To handle this part, data cleaning is done. It involves handling of missing data, noisy data etc.

(a). Missing Data:

This situation arises when some data is missing in the data. It can be handled in various ways.

Some of them are:

Ignore the tuples:

This approach is suitable only when the dataset we have is quite large and multiple values are missing within a tuple.

Fill the Missing values:

There are various ways to do this task. You can choose to fill the missing values manually, by attribute mean or the most probable value.

(b). Noisy Data:

Noisy data is a meaningless data that can't be interpreted by machines.It can be generated due to faulty data collection, data entry errors etc. It can be handled in following ways :

Binning Method:

This method works on sorted data in order to smooth it. The whole data is divided into segments of equal size and then various methods are performed to complete the task. Each segmented is handled separately. One can replace all data in a segment by its mean or boundary values can be used to complete the task.

Regression:

Here data can be made smooth by fitting it to a regression function.The regression used may be linear (having one independent variable) or multiple (having multiple independent variables).

Clustering:

This approach groups the similar data in a cluster. The outliers may be undetected or it will fall outside the clusters.

2. Data Transformation:

This step is taken in order to transform the data in appropriate forms suitable for mining process. This involves following ways:

Normalization:

It is done in order to scale the data values in a specified range (-1.0 to 1.0 or 0.0 to 1.0)

Attribute Selection:

In this strategy, new attributes are constructed from the given set of attributes to help the mining process.

Discretization:

This is done to replace the raw values of numeric attribute by interval levels or conceptual levels.

Concept Hierarchy Generation:

Here attributes are converted from lower level to higher level in hierarchy. For Example-The attribute "city" can be converted to "country".

3. Data Reduction:

Data reduction is a crucial step in the data mining process that involves reducing the size of the dataset while preserving the important information. This is done to improve the efficiency of data analysis and to avoid overfitting of the model. Some common steps involved in data reduction are:

Feature Selection: This involves selecting a subset of relevant features from the dataset. Feature selection is often performed to remove irrelevant or redundant features from the dataset. It can be done using various techniques such as correlation analysis, mutual information, and principal component analysis (PCA).

Feature Extraction: This involves transforming the data into a lower-dimensional space while preserving the important information. Feature extraction is often used when the original features are high-dimensional and complex. It can be done using techniques such as PCA, linear discriminant analysis (LDA), and non-negative matrix factorization (NMF).

Sampling: This involves selecting a subset of data points from the dataset. Sampling is often used to reduce the size of the dataset while preserving the important information. It can be done using techniques such as random sampling, stratified sampling, and systematic sampling.

Clustering: This involves grouping similar data points together into clusters. Clustering is often used to reduce the size of the dataset by replacing similar data points with a representative centroid. It can be done using techniques such as k-means, hierarchical clustering, and density-based clustering.

Compression: This involves compressing the dataset while preserving the important information. Compression is often used to reduce the size of the dataset for storage and transmission purposes. It can be done using techniques such as wavelet compression, JPEG compression, and gzip compression.

Binning can also be used as a ***discretization technique***. Here discretization refers to the process of converting or partitioning continuous attributes, features or variables to discretized or nominal attributes/features/variables/intervals.

For example, attribute values can be discretized by applying *equal-width* or *equal-frequency* binning, and then replacing each bin value by the bin mean or median, as in smoothing by bin means or smoothing by bin medians, respectively. Then the continuous values can be converted to a nominal or discretized value which is same as the value of their corresponding bin.

Handling Missing Data in Pandas

Introduction

In Pandas, missing data refers to unavailable or empty values in a dataset.

Missing values are represented as:

NaN # Not a Number

Missing data may occur due to:

- Incomplete data collection
- Human error
- System failure
- Data corruption

Handling missing data is important for accurate data analysis and machine learning.

Detecting Missing Data

1. isnull()

Used to identify missing values.

Syntax

```
DataFrame.isnull()
```

Example

```
import pandas as pd
 import numpy as np

data = {
   "Name": ["Ram", "Sita", "Hari"],
   "Age": [20, np.nan, 22]
}

df = pd.DataFrame(data)

print(df.isnull())
```

Output

```
    Name    Age
0  False  False
1  False   True
2  False  False
```

Counting Missing Values

2. isnull().sum()

Counts total missing values in each column.

Syntax

```
DataFrame.isnull().sum()
```

Example

```
print(df.isnull().sum())
```

Output

```
Name    0
 Age    1
 dtype: int64
```

Removing Missing Data

3. dropna()

Removes rows containing missing values.

Syntax

```
DataFrame.dropna()
```

Example

```
new_df = df.dropna()

 print(new_df)
```

Output

```
    Name   Age
0   Ram  20.0
2  Hari  22.0
```

Filling Missing Data

4. fillna()

Replaces missing values with specified values.

Syntax

```
DataFrame.fillna(value)
```

Example

```
df["Age"] = df["Age"].fillna(21)

 print(df)
```

Output

```
    Name   Age
0   Ram  20.0
1  Sita  21.0
2  Hari  22.0
```

Filling Missing Values with Mean

Mean Method

Syntax

```
DataFrame["column_name"].fillna(
    DataFrame["column_name"].mean()
 )
```

Example

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Forward Fill Method

5. ffill()

Copies previous value forward.

Syntax

```
DataFrame.ffill()
```

Example

```
df.ffill()
```

Backward Fill Method

6. bfill()

Copies next value backward.

Syntax

```
DataFrame.bfill()
```

Example

```
df.bfill()
```

Complete Practical Program

```
import pandas as pd
 import numpy as np

# Creating DataFrame
data = {
   "Name": ["Ram", "Sita", "Hari", "Gita"],
   "Age": [20, np.nan, 22, 24],
   "Salary": [30000, 35000, np.nan, 40000]
}

df = pd.DataFrame(data)

print("Original DataFrame")
print(df)

# Detect missing values
print("\nMissing Values")
print(df.isnull())

# Count missing values
print("\nTotal Missing Values")
print(df.isnull().sum())

# Fill missing Age with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Fill missing Salary with 0
df["Salary"] = df["Salary"].fillna(0)

print("\nUpdated DataFrame")
print(df)

# Remove missing values
print("\nAfter Removing Missing Values")
print(df.dropna())
```

Python dictionary and perform two operations on it:

Add a new key-value pair and update an existing value.

```
# Create a dictionary
student = {
   "name": "Ayush",
   "age": 14,
   "grade": "7"
}
# Display original dictionary
print("Original Dictionary:")
print(student)
# Add a new key-value pair
student["city"] = "Mahendranagar"
student["School"]="SARC School"
# Update an existing value
student["age"] = 13
student["grade"]=8
# Display updated dictionary
print("\nUpdated Dictionary:")
print(student)
```

Practical Example Based on Syllabus

Combining & Merging Data, Reshaping, Pivoting, GroupBy, Aggregation, and String Manipulation

This practical covers topics from Pandas syllabus:

- Combining and Merging Data Sets
- Data Transformation
- String Manipulation
- GroupBy Mechanics
- Data Aggregation
- Pivot Tables and Cross Tabulation

Practical Question (10 Marks)

A company has two datasets:

Employee Data

EmpID	Name	Department
101	ram sharma	IT
102	sita thapa	HR
103	hari kc	IT
104	gita rai	Finance

Salary Data

EmpID	Salary
101	50000
102	45000
103	55000
104	48000

Write a Python program using Pandas to:

1. Create both DataFrames.
2. Merge the datasets using EmpID.
3. Convert employee names into uppercase.
4. Add 10% bonus to salary.
5. Find average salary department-wise using groupby().
6. Create a pivot table showing average salary by department.

Solution

```
import pandas as pd
# Employee DataFrame
employee = pd.DataFrame({
    "EmpID": [101, 102, 103, 104],
    "Name": ["ram sharma", "sita thapa", "hari kc", "gita rai"],
    "Department": ["IT", "HR", "IT", "Finance"]
})
# Salary DataFrame
salary = pd.DataFrame({
    "EmpID": [101, 102, 103, 104],
    "Salary": [50000, 45000, 55000, 48000]
})
# Merge DataFrames
df = pd.merge(employee, salary, on="EmpID")
print("Merged DataFrame")
print(df)
# String Manipulation
df["Name"] = df["Name"].str.upper()
# Data Transformation
df["BonusSalary"] = df["Salary"] + (df["Salary"] * 0.10)
print("\nUpdated DataFrame")
print(df)
# GroupBy and Aggregation
group_data = df.groupby("Department")["Salary"].mean()
print("\nAverage Salary Department-wise")
print(group_data)
# Pivot Table
pivot = pd.pivot_table(
    df,
    values="Salary",
    index="Department",
    aggfunc="mean"
)
print("\nPivot Table")
print(pivot)
```

Simple Pivot Table in Pandas

Theory

A **Pivot Table** in Pandas is used to summarize data.

It helps to:

- Calculate totals
- Find averages
- Group similar data
- Create summary reports

Syntax

```
pd.pivot_table(
    dataframe,
    values="column_name",
    index="column_name",
    aggfunc="function"
)
```

Meaning of Parameters

Parameter	Meaning
values	Column to calculate
index	Column used for grouping
aggfunc	Function like sum, mean, max

Simple Practical Example

Question

Create a pivot table to find total sales department-wise.

Program

```
import pandas as pd

# Creating DataFrame
data = {
    "Department": ["IT", "HR", "IT", "Finance", "HR"],
    "Sales": [5000, 3000, 7000, 4000, 3500]
}
df = pd.DataFrame(data)
print("Original DataFrame")
print(df)
# Creating Pivot Table
pivot = pd.pivot_table(
    df,
    values="Sales",
    index="Department",
    aggfunc="sum"
)
print("\nPivot Table")
print(pivot)
```

Output

Explanation

- IT sales = 5000 + 7000 = 12000
- HR sales = 3000 + 3500 = 6500
- Finance sales = 4000

The pivot table groups departments and calculates total sales.

Common Aggregation Functions

Function	Purpose
sum	Total
mean	Average
max	Maximum value
min	Minimum value
count	Count values

Example with Average

```
pivot = pd.pivot_table(
    df,
    values="Sales",
    index="Department",
    aggfunc="mean"
)
print(pivot)
```

This gives average sales department-wise.

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data (attributes) and code (methods). Unlike procedural programming, which focuses on functions and logic, OOP focuses on the data objects that developers want to manipulate.

1. The Fundamental Building Blocks

Before diving into the pillars, we must understand the relationship between a Class and an Object:

* **Class:** A blueprint or template for creating objects. It defines the properties and behaviors that its objects will have.
* **Object:** An instance of a class. It is a real-world entity with a specific state and behavior.
  * *Example:* If **Car** is a class, then your **Tesla Model 3** is the object.

2. The Four Pillars of OOP

The strength of OOP lies in these four principles, designed to make code reusable, scalable, and secure.

A. Abstraction

Abstraction is the process of **hiding complex implementation details** and showing only the necessary features of an object. It reduces complexity by allowing the user to interact with a high-level interface without needing to understand the internal logic.

* **Example:** When you press "Play" on a remote, you don't need to know how the circuitry works; you only care about the result.

B. Encapsulation

Encapsulation is the **bundling of data and methods** into a single unit (a class) and restricting direct access to some of the object's components. This is typically achieved using "private" variables and "public" getter/setter methods.

* **Goal:** To protect the internal state of an object from unintended interference.

C. Inheritance

Inheritance allows a new class (Subclass/Child) to **acquire the properties and methods** of an existing class (Superclass/Parent).

* **Benefits:** It promotes **code reusability** and establishes a natural hierarchy.
* **Example:** A Dog class can inherit from an Animal class.

D. Polymorphism

The word means "many forms." In programming, it allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the object.

* **Method Overloading:** Same method name, different parameters (Compile-time).
* **Method Overriding:** A child class provides a specific implementation of a method already defined in its parent class (Run-time).

3. Advantages of OOP

* **Reusability:** Through inheritance, we can use existing code without rewriting it.
* **Data Redundancy:** OOP inheritance helps in avoiding duplication of data.
* **Maintenance:** Since code is modular (divided into classes), it is easier to update and maintain.
* **Security:** Encapsulation ensures that data is not accessed or modified by external code accidentally.

> **Summary Table for Quick Reference**
> | Feature | Description | Key Benefit |
> |---|---|---|
> | **Abstraction** | Hiding internal details. | Simplicity |
> | **Encapsulation** | Wrapping data in a unit. | Security/Privacy |
> | **Inheritance** | Deriving new classes from old. | Reusability |
> | **Polymorphism** | Performing tasks in different ways. | Flexibility